

La solution : la conception d'instructions stratégiques

Instructions concernant l'utilisation des outils

Référence : [Documentation ADK - Faire référence à un outil dans les instructions d'un agent](#)

Extrait de la documentation ADK :

"Dans les instructions d'un agent, vous pouvez faire directement référence à un outil en utilisant le nom de sa fonction... Il est essentiel d'indiquer clairement à l'agent comment gérer les différentes valeurs de retour qu'un outil peut produire."

Qu'apportent les instructions stratégiques ?

-  **Guide de sélection des outils** : quand utiliser quel outil
-  **Gestion des exceptions** : comment réagir aux différents types d'exceptions
-  **Logique séquentielle** : dans quel ordre appeler les outils
-  **Intégration des résultats** : comment utiliser les résultats des outils
-  **Procédures d'escalade** : que faire lorsque les outils ne peuvent pas résoudre le problème

Principe clé : Les instructions doivent indiquer **quand** et **comment** utiliser les outils, et pas seulement **que** faire avec eux (les docstrings s'en chargent).

Concepts fondamentaux

1. Modèles d'instructions pour l'utilisation des outils

Référence : [Documentation ADK - Faire référence à un outil dans les instructions d'un agent](#)

Modèle 1 : Guide de sélection des outils

Dites à l'agent quand il doit utiliser chaque outil :

```
instruction=""
Tu es un agent du service client.

Guide de sélection des outils :
- Utilise check_order_status lorsque le client s'informe sur sa commande.
- Utilise process_refund lorsqu'un client demande un remboursement.
- Utilise lookup_customer lorsque tu as besoin d'informations sur le compte d'un client.

Vérifie toujours les informations avant d'agir.
""
```

Modèle 2 : Gestion des résultats

Dites à l'agent comment il doit gérer les différents résultats :

```
instruction="""
Lorsque tu utilises check_order_status :
1. Appelle l'outil avec l'ID de commande.
2. Si l'état est "success" :
    - Informe le client de l'état actuel de sa commande.
    - Si les informations de suivi sont disponibles, fournis-les.
3. Si le statut est "error" avec error_type "not_found" :
    - Demande au client de vérifier l'ID de la commande.
    - Propose-lui d'utiliser son adresse e-mail à la place.
4. Si le statut est "error" avec error_type "invalid_format" :
    - Explique que les ID de commande doivent commencer par "ORD".
    - Demande le format correct.
"""
```

Modèle 3 : Utilisation séquentielle des outils

Guidez l'agent à travers les workflows comportant plusieurs étapes :

```
instruction="""
Pour les demandes de remboursement :
1. Commence par utiliser check_order_status pour vérifier que la commande existe.
2. Si la commande est introuvable, arrête de la chercher et demande au client de te
fournir le bon ID de commande.
3. Si tu trouves la commande, utilise process_refund avec order_id et reason.
4. Si le remboursement aboutit, confirme son montant et le numéro de référence.
5. Si le remboursement échoue, explique pourquoi et propose d'escalader le problème à
un superviseur.
"""
```

Exemple complet :

```
from google.adk.agents import LlmAgent

agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="""
    Tu es un agent du service client serviable.

    ## Sélection des outils
    - Utilise check_order_status lorsque le client demande l'état de sa commande.
    - Utilise process_refund pour les demandes de remboursement.
    - Utilise escalate_to_supervisor pour les problèmes complexes.

    ## Workflows

    ### Demande d'état de la commande :
    1. Salue le client.
    2. Utilise check_order_status avec l'ID de commande.
    3. En cas de succès, indique clairement l'état de la commande.
    4. En cas d'erreur, oriente le client vers les informations appropriées.
    """
```

```

### Demande de remboursement :
1. Fais preuve d'empathie.
2. Vérifie d'abord la commande avec check_order_status.
3. Si une commande existe, utilise process_refund.
4. Confirme les détails du remboursement au client.

## Gestion des exceptions
- Erreurs "not_found" : demande au client de vérifier les informations.
- Erreurs "invalid_format" : explique le format correct.
- Erreurs inattendues : utilise escalate_to_supervisor.

Reste poli et professionnel en toute circonstance.
"""
tools=[check_order_status, process_refund, escalate_to_supervisor]
)

```

Points à noter :

- **Sections organisées** : sélection des outils, workflows, gestion des exceptions
- **Conseils spécifiques** : quand utiliser chaque outil
- **Étape par étape** : instructions séquentielles claires
- **Gestion des exceptions** : que faire pour chaque type d'erreur

Visualisation du workflow séquentiel :

```

graph TD
    A[Requête utilisateur : remboursement de commande] --> B{Raisonnement de l'agent}
    B --> C[Outil : check_order_status]
    C --> D{Commande trouvée ?}
    D -->|Non| E[Réponse d'erreur :<br/>Demander à l'utilisateur de vérifier]
    D -->|Oui| F{État de la commande ?}
    F -->|Livrée| G[Outil : process_refund]
    F -->|En cours de traitement| H[Réponse d'erreur :<br/>Remboursement impossible pour le moment]
    F -->|Annulée| I[Réponse d'erreur :<br/>Déjà annulée]
    G --> J{Remboursement réussi ?}
    J -->|Oui| K[Réponse positive :<br/>Confirmer à l'utilisateur]
    J -->|Non| L[Outil : escalate_to_supervisor]
    L --> M[Proposer d'escalader]
    M --> N[Superviseur affecté]

    style E fill:#ffc000
    style H fill:#ff9900
    style I fill:#ff9900
    style K fill:#ccffcc
    style L fill:#ff6600

```

Les instructions stratégiques guident les agents à travers des workflows complexes, avec une gestion appropriée des exceptions.

2. Gestion des exceptions dans les instructions

Référence : [Documentation ADK - Faire référence à un outil dans les instructions d'un agent](#)

Extrait de la documentation ADK :

"Il est essentiel d'indiquer clairement à l'agent comment gérer les différentes valeurs de retour qu'un outil peut produire. Par exemple, dans le cas où un outil renverrait un message d'erreur, vos instructions doivent préciser si l'agent doit réessayer l'opération, abandonner la tâche ou demander des informations supplémentaires à l'utilisateur."

Stratégies de gestion des exceptions :

Stratégie 1 : Réessayer ou abandonner

```
instruction="""
```

```
Consignes de gestion des exceptions :
```

```
Si check_order_status renvoie "temporary_failure" :
```

- Explique au client qu'il y a un problème.
- Réessaie au bout de quelques instants.
- Si la deuxième tentative échoue, présente tes excuses et invite le client à réessayer plus tard.

```
Si check_order_status renvoie "not_found" :
```

- Ne réessaie PAS avec le même ID de commande.
- Demande au client de vérifier l'ID de commande.
- Propose d'autres méthodes de recherche (adresse e-mail).

```
Si check_order_status renvoie "permission_denied" :
```

- Ne réessaie PAS et ne cherche pas non plus à contourner le problème.
 - Présente tes excuses et utilise immédiatement `escalate_to_supervisor`.
- ```
"""
```

#### Stratégie 2 : Actions propres à chaque erreur

```
def lookup_order(order_id: str) -> dict:
```

```
 """Recherche les informations sur la commande.
```

```
 Renvoie :
```

```
 dict : informations sur la commande.
```

```
 En cas de succès : {'status': 'success', 'order': {...}}
```

```
 En cas d'erreur : {'status': 'error', 'error_type':
```

```
'not_found'/'invalid_format'/'permission_denied'}
```

```
 """
```

```
 # Implémentation avec des types d'erreurs spécifiques
```

```
 pass
```

```
agent = LlmAgent(
```

```
 instruction="""
```

```
 Lorsque tu utilises lookup_order :
```

```
 Si error_type est "not_found" :
```

- Dis : "Je n'ai pas trouvé cette commande." Pourriez-vous vérifier l'ID de commande ?"
- Propose une recherche par adresse e-mail ou numéro de téléphone.
- Ne formule PAS d'hypothèses.

Si error\_type est "invalid\_format" :

- Dis : "Les ID de commande doivent être au format ORD-12345."
- Demande au client de fournir le format correct.
- Donne un exemple.

Si error\_type est "permission\_denied" :

- Dis : "Je n'ai pas accès à cette commande. Je vais vous mettre en relation avec un superviseur."

- Utilise immédiatement escalate\_to\_supervisor.
- Ne tente PAS d'approches alternatives.

```
""",
tools=[lookup_order, escalate_to_supervisor]
```

)

### Stratégie 3 : Dégradation élégante

```
instruction="""
```

Ordre de priorité pour la recherche d'un client :

1. Commence par essayer lookup\_by\_order\_id (ce qu'il y a de plus rapide et de plus direct).
2. Si c'est un échec et que la réponse "not\_found" est renvoyée, essaie lookup\_by\_email.
3. Si cela ne fonctionne pas, essaie lookup\_by\_phone.
4. Si toutes les recherches échouent, utilise escalate\_to\_supervisor.

À chaque étape :

- Explique ce que tu fais.
  - Demande poliment des renseignements.
  - N'agis jamais sans données valides.
- ```
"""
```

Principe clé : différents types d'erreur nécessitent différentes stratégies de gestion. Précisez exactement ce que l'agent doit faire dans chaque situation.

Arbre de décision pour la gestion des exceptions :

flowchart TD

```

A[Erreur d'outil reçue] --> B{Type d'erreur ?}
B -->|temporary_failure| C[En faire part<br/>au client]
C --> D[Réessayer une fois]
D --> E{Succès ?}
E -->|Oui| F[Continuer le workflow]
E -->|Non| G[S'excuser et demander<br/>de réessayer plus tard]

B -->|not_found| H[Demander à l'utilisateur de<br/>vérifier l'entrée]
H --> I[Proposer des méthodes<br/>de recherche alternatives]

B -->|invalid_format| J[Expliquer<br/>le bon format]
```

```

J --> K[Fournir des<br/>exemples de format]

B -->|permission_denied| L[Escalader<br/>immédiatement]

B -->|system_error| M[S'excuser<br/>sincèrement]
M --> N[Enregistrer l'erreur<br/>pour l'équipe]
N --> O[Proposer de<br/>rappeler]

style L fill:#ffcccc
style G fill:#fff9cc
style H fill:#fff9cc
style J fill:#fff9cc
style M fill:#ffcccc
style F fill:#ccffcc

```

La réponse de l'agent n'est pas la même selon le type d'erreur qu'il rencontre. Indiquez exactement ce qu'il doit faire dans les instructions.

3. Modèle agent-as-tool (aperçu)

Couverture complète : prochains cours sur la coordination entre plusieurs agents

Qu'est-ce qu'un modèle agent-as-tool (agent comme outil) ?

Au lieu d'écrire un outil de fonction, vous pouvez utiliser un autre agent spécialisé comme outil. Cela permet à l'agent principal de déléguer des sous-tâches complexes à des agents spécialisés.

Quand utiliser le modèle agent-as-tool ?

- ☒ La **sous-tâche nécessite un raisonnement spécialisé** (et pas seulement une logique prédéfinie).
- ☒ Des **instructions différentes sont nécessaires** pour la sous-tâche.
- ☒ La sous-tâche comporte des **workflows complexes**.

Prenons un exemple simple :

```

from google.adk.agents import LlmAgent
from google.adk.tools.agent_tool import AgentTool

# Agent spécialisé dans l'assistance technique
tech_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='tech_support',
    instruction="Tu es un spécialiste de l'assistance technique. Diagnostic les problèmes et propose des solutions de dépannage."
)

# L'agent principal utilise un spécialiste comme outil
main_agent = LlmAgent(

```

```

    model='gemini-2.5-flash',
    name='customer_service',
    instruction="Transmets les problèmes techniques à l'outil tech_support. Réponds
directement aux questions générales.",
    tools=[AgentTool(agent=tech_agent)]
)

```

Différences majeures :

Aspect	Outil de fonction	Agent-as-tool
Implémentations	Logique prédéfinie	Raisonnement et prise de décisions
Application idéale	Calculs, recherches, appels d'API	Sous-tâches complexes nécessitant un jugement
Exemple	<code>calculate_shipping()</code>	Spécialiste de l'assistance technique

Remarque : La coordination entre plusieurs agents, les modèles de délégation et l'utilisation agent-as-tool avancée sont abordés dans les prochaines leçons sur les modèles multi-agents.

4. Bonnes pratiques pour la coordination des outils

Référence : [Documentation ADK - Définir des fonctions d'outil efficaces](#)

Extrait de la documentation ADK :

"En respectant ces consignes, vous offrez au LLM la clarté et la structure dont il a besoin pour utiliser efficacement vos outils de fonction personnalisés. En conséquence, le comportement des agents sera plus performant et plus fiable."

Récapitulatif des bonnes pratiques :

1. Conception de fonctions

```

# ✓ Bonne pratique : outils ciblés et dédiés
def check_order_status(order_id: str) -> dict:
    """Vérifie uniquement l'état de la commande."""
    pass

def cancel_order(order_id: str) -> dict:
    """Annule la commande uniquement."""
    pass

# ✗ Mauvaise pratique : outils multifonctions qui en font trop
def manage_order(order_id: str, action: str, params: dict) -> dict:
    """Fait tout : vérification, annulation, mise à jour, etc."""

```

```
pass
```

2. Dénomination claire

 Bonne pratique : structure verbe-nom, descriptive

```
get_customer_profile()
calculate_shipping_cost()
send_confirmation_email()
validate_coupon_code()
```

 Mauvaise pratique : vague ou imprécise

```
process_data()
handle_request()
do_thing()
execute()
```

3. Docstrings complètes

```
def process_refund(order_id: str, reason: str) -> dict:
    """Traite le remboursement d'une commande client.
```

N'utilise cet outil qu'APRÈS avoir vérifié l'existence de la commande et son éligibilité au remboursement.

Args:

```
    order_id (str) : l'identifiant de la commande (format : ORD-12345)
    reason (str) : motif de la demande de remboursement par le client
```

Renvoie :

```
    dict : résultat du traitement du remboursement
        En cas de succès : {
            'status': 'success',
            'refund_amount': 99.99,
            'refund_id': 'REF-67890',
            'estimated_days': 5
        }
        En cas d'erreur : {
            'status': 'error',
            'error_type': 'not_eligible'/'already_refunded'/'not_found',
            'error_message': 'explication lisible par l'humain'
        }
    """
```

```
pass
```

4. Format de retour cohérent

 Bonne pratique : modèle cohérent basé sur l'état

```
def tool_1() -> dict:
    return {"status": "success", "data": value}
    # ou
    return {"status": "error", "error_type": "specific_type", "error_message":
"explanation"}
```

```
def tool_2() -> dict:
```

```

    return {"status": "success", "result": value}
# ou
    return {"status": "error", "error_type": "specific_type", "error_message":
"explanation"}

# ❌ Mauvaise pratique : valeurs de retour incohérentes
def tool_1() -> dict:
    return {"ok": True, "value": value} # Format différent

def tool_2() -> str:
    return "success" or "error: message" # Renvoie une chaîne au lieu d'un
dictionnaire

```

5. Instructions stratégiques

```

instruction=""
## Consignes d'utilisation des outils

### Sélection des outils
[Quand utiliser quel outil]

### Workflows
[Procédures étape par étape]

### Gestion des exceptions
[Que faire pour chaque type d'erreur]

### Escalade
[Quand et comment escalader]
"""

```

Suivez les points de la checklist :

- Les noms d'outils sont descriptifs (verbe-nom).
- Tous les paramètres possèdent des indications de type.
- Les docstrings expliquent quoi, quand, les arguments et les valeurs de retour.
- Les valeurs de retour incluent la clé d'état.
- Les messages d'erreur sont faciles à comprendre.
- Les instructions désignent les outils par leur nom.
- Les instructions gèrent les cas de succès et tous les types d'erreurs.
- Les workflows séquentiels sont clairement définis.
- Les procédures d'escalade sont spécifiées.

5. Outils de coordination avec l'état (aperçu)

Lien avec la leçon 4 : Gestion de l'état et de la mémoire

Les outils peuvent lire et écrire l'état de session pour coordonner les workflows en plusieurs étapes, suivre l'utilisation et fournir un comportement personnalisé. `tool_context` sera entièrement traité dans les leçons suivantes, mais voici ce qu'il est possible de faire :

Cas d'utilisation de l'état dans les outils :

✓ **Workflows en plusieurs étapes** : les outils enregistrent les résultats intermédiaires dans l'état pour l'outil suivant.

✓ **Suivi de l'utilisation** : comptage programmatique des appels d'outils.

✓ **Personnalisation** : les outils lisent les préférences utilisateur dans l'espace de noms `user::`.

✓ **Logique conditionnelle** : les outils vérifient l'état pour déterminer le comportement.

Exemple : Workflow en plusieurs étapes avec coordination d'état

```
def check_order_status(order_id: str, ctx) -> dict:
    """Vérifie l'état de la commande et enregistre cette information dans l'état pour
    les autres outils.
```

Args:

order_id : l'ID de commande à vérifier

ctx : contexte de l'outil (donne accès à `session.state`)

Renvoie :

dict : informations sur l'état de la commande

Remarque : Le paramètre `ctx` (`tool_context`) est traité dans la leçon 6.

"""

Rechercher la commande

order = database.get_order(order_id)

Enregistrer les informations sur la commande dans l'état de session pour que
d'autres outils puissent les utiliser

ctx.session.state['current_order_id'] = order_id

ctx.session.state['current_order_status'] = order['status']

Suivre l'utilisation avec l'état de session

count = ctx.session.state.get('orders_checked', 0)

ctx.session.state['orders_checked'] = count + 1

return {

"status": "success",

"order_id": order_id,

"order_status": order['status'],

"details": order

}

```
def process_refund(order_id: str, reason: str, ctx) -> dict:
```

"""Traite les remboursements en utilisant les données de commande enregistrées
 dans l'état.

Utilise l'état de `check_order_status` pour éviter les recherches redondantes.

"""

Lire l'état de la commande à partir de l'état (enregistré par l'outil précédent)

saved_order_id = ctx.session.state.get('current_order_id')

saved_status = ctx.session.state.get('current_order_status')

```

    # Vérifier qu'il s'agit bien de la même commande que celle que nous venons de
    contrôler
    if saved_order_id == order_id and saved_status:
        # Utiliser l'état mis en cache au lieu d'effectuer une nouvelle requête
        if saved_status != 'delivered':
            return {
                "status": "error",
                "error_type": "cannot_refund",
                "error_message": f"L'état de la commande est '{saved_status}', ce qui
signifie qu'elle n'a pas été livrée."
            }

    # Traiter le remboursement...
    refund_id = process_refund_in_system(order_id, reason)

    # Suivre le nombre de remboursements
    count = ctx.session.state.get('refunds_processed', 0)
    ctx.session.state['refunds_processed'] = count + 1

    return {
        "status": "success",
        "refund_id": refund_id
    }

```

Espaces de noms d'état dans les outils (de la leçon 4) :

```

# Données temporaires (supprimées après le tour)
ctx.session.state['temp:validation_result'] = result

# Données de session (persistent d'un tour à l'autre dans la conversation)
ctx.session.state['orders_checked'] = 5

# Préférences utilisateur (persistent d'une session à l'autre)
ctx.session.state['user:preferred_currency'] = 'USD'

# Configuration globale (partagée par tous les utilisateurs)
ctx.session.state['app:max_refund_amount'] = 500.00

```

Fonctionnalités proposées :

- **Coordination des outils** : un outil enregistre les données, un autre les lit.
- **Suivi précis** : accès programmatique au décompte – `if state.get('failed_attempts') >= 3:`
- **Personnalisation** : les outils s'adaptent en fonction des préférences `user` :.
- **Workflows conditionnels** : les outils prennent des décisions en fonction des valeurs d'état.

Aperçu : L'intégration complète avec `tool_context`, l'accès à l'état dans les outils personnalisés et les modèles de coordination d'état avancés sont abordés dans la **leçon 6** : **Rappels et flux de contrôle**.

Exemple pratique

Créer un agent du service client avec des outils coordonnés

Objectif : Créer un agent du service client qui gère les demandes de renseignements sur les commandes, traite les remboursements et escalade les problèmes complexes, avec une gestion complète des exceptions et une coordination stratégique des outils.

Étape 1 : Créer le projet

```
adk create customer_support
cd customer_support
```

Fonction :

- Crée un nouveau répertoire de projet ADK
 - Établit une structure d'agent de base
 - Crée `agent.py` avec le code par défaut
-

Étape 2 : Écrire l'agent

Remplacez le contenu de `agent.py` par :

```
"""
Agent du service client avec des outils coordonnés
Illustre une combinaison stratégique d'outils, une gestion efficace des exceptions et
le modèle agent-as-tool.

Référence : https://google.github.io/adk-docs/tools-custom/
"""

from google.adk.agents import LlmAgent

# Base de données simulée
ORDERS_DB = {
    "ORD123": {"status": "shipped", "total": 99.99, "customer": "john@email.com"},
    "ORD456": {"status": "processing", "total": 149.99, "customer": "jane@email.com"},
    "ORD789": {"status": "delivered", "total": 249.99, "customer": "bob@email.com"},
}

# Outil 1 : vérifier l'état de la commande
def check_order_status(order_id: str) -> dict:
    """Vérifie l'état actuel de la commande d'un client.

    Utilise cette option lorsqu'un client s'informe sur l'état ou la livraison de sa
```

commande.

Args:

order_id (str) : l'identifiant de la commande (par exemple, "ORD123").

Renvoie :

dict : informations sur l'état de la commande.

En cas de succès : {'status': 'success', 'order_status': '...', 'details': {...}}

En cas d'erreur : {'status': 'error', 'error_type': 'not_found'/'invalid_format'}

Valider le format

```
if not order_id.startswith("ORD"):
    return {
        "status": "error",
        "error_type": "invalid_format",
        "error_message": "Les ID de commande doivent commencer par 'ORD' (par exemple, ORD123)"
    }
```

Rechercher la commande

```
if order_id not in ORDERS_DB:
    return {
        "status": "error",
        "error_type": "not_found",
        "error_message": f"Commande {order_id} introuvable dans le système"
    }
```

Réussite

```
order = ORDERS_DB[order_id]
return {
    "status": "success",
    "order_id": order_id,
    "order_status": order["status"],
    "details": order
}
```

Outil 2 : traiter le remboursement

```
def process_refund(order_id: str, reason: str) -> dict:
    """Traite une demande de remboursement pour une commande.
```

Utilise ceci UNIQUEMENT après avoir vérifié l'existence de la commande avec check_order_status.

Args:

order_id (str) : ID de la commande à rembourser.

reason (str) : motif du client souhaitant être remboursé.

Renvoie :

dict : résultat du traitement du remboursement.

En cas de succès : {'status': 'success', 'refund_amount': X, 'reference': 'REF###'}

En cas d'erreur : {'status': 'error', 'error_type':

```

'order_not_found'/'cannot_refund'}
"""

# Vérifier si une commande existe
if order_id not in ORDERS_DB:
    return {
        "status": "error",
        "error_type": "order_not_found",
        "error_message": f"Impossible de traiter le remboursement - Commande
{order_id} introuvable"
    }

order = ORDERS_DB[order_id]

# Vérifier si la commande est éligible à un remboursement
if order["status"] == "delivered":
    # Simuler un remboursement réussi
    return {
        "status": "success",
        "refund_amount": order["total"],
        "reference": f"REF{order_id[3:]}",
        "estimated_days": 5,
        "message": "Remboursement traité avec succès"
    }
else:
    # Impossible de rembourser les commandes pas encore livrées
    return {
        "status": "error",
        "error_type": "cannot_refund",
        "error_message": f"Impossible de rembourser une commande dont l'état est
'{order['status']}'". Seules les commandes livrées peuvent être remboursées."
    }

# Outil 3 : escalader au superviseur
def escalate_to_supervisor(issue_summary: str, order_id: str) -> dict:
    """Escalade les problèmes complexes à un superviseur humain.

    Utilise cette méthode lorsque tu ne parviens pas à résoudre le problème du client
    avec les outils à ta disposition
    ou lorsque le client demande explicitement à parler à un superviseur.

    Args:
        issue_summary (str) : bref résumé du problème (1-2 phrases).
        order_id (str) : ID de la commande associée, le cas échéant.

    Renvoie :
        dict : confirmation d'escalade.
        Renvoie toujours : {'status': 'success', 'ticket_id': 'TICKET###'}
    """
    # Générer un ID de demande
    ticket_id = f"TICKET{hash(issue_summary) % 10000:04d}"

    return {
        "status": "success",
        "ticket_id": ticket_id,

```

```

        "message": "Problème escaladé au superviseur",
        "estimated_response": "dans les deux heures",
        "order_id": order_id if order_id else "N/A"
    }

# Créer un agent du service client avec des instructions stratégiques
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='customer_support_agent',
    description='Traite les demandes des clients concernant les commandes et les remboursements avec une gestion complète des exceptions.',
    instruction="""
    Tu es un agent du service client serviable et empathique pour une entreprise d'e-commerce.

    # Tes capacités

    Tu disposes de trois outils :
    1. check_order_status(order_id) pour vérifier l'état d'une commande.
    2. process_refund(order_id, reason) pour traiter les demandes de remboursement.
    3. escalate_to_supervisor(issue_summary, order_id) pour escalader les problèmes complexes.

    # Consignes concernant le workflow

    ## Pour les questions portant sur l'état d'une commande :
    1. Salue chaleureusement le client.
    2. Utilise check_order_status avec l'ID de commande qu'il fournit.
    3. Gère le résultat :
        - Si status='success', fournis des explications claires et détaillées sur l'état.
        - Si error_type='not_found', demande poliment au client de vérifier l'ID de la commande.
        - Si error_type='invalid_format', explique que les ID de commande commencent par "ORD" (exemple : ORD123).

    ## Pour les demandes de remboursement :
    1. Fais preuve d'empathie face à la situation.
    2. COMMENCE par utiliser check_order_status pour vérifier que la commande existe.
    3. Si la commande est introuvable, le remboursement ne peut pas être effectué. Demande au client de vérifier l'ID de la commande.
    4. Si la commande existe, utilise process_refund avec l'ID de la commande et le motif du client.
    5. Gère le résultat du remboursement :
        - Si status='success', confirme le remboursement avec le numéro de référence, le montant et le délai.
        - Si error_type='cannot_refund', explique pourquoi (état de la commande) et propose des alternatives ou une procédure d'escalade.
        - Si error_type='order_not_found' (cela ne devrait pas se produire si l'étape 2 a abouti), présente tes excuses et vérifie.

    ## Stratégie de gestion des exceptions :

    Pour les erreurs "not_found" :

```

- Demande au client de vérifier l'ID de la commande.
- Propose une recherche par adresse e-mail le cas échéant.
- Sois patient et serviable.

Pour les erreurs "invalid_format" :

- Explique poliment quel est le bon format : "ORD" suivi de plusieurs chiffres.
- Donne un exemple : ORD123.
- Demande au client de fournir l'ID de la commande au format correct.

Pour les erreurs "cannot_refund" :

- Explique clairement la règle (seules les commandes livrées peuvent être remboursées).
- Fais preuve d'empathie face à l'agacement du client.
- Propose d'escalader le problème à ton supérieur s'il veut qu'une exception soit faite.

Quand escalader :

Utilise `escalate_to_supervisor` dans les situations suivantes :

- Le client est agacé ou en colère et demande à parler au superviseur.
- Le problème ne peut pas être résolu avec les outils à ta disposition.
- Le client souhaite qu'une exception soit faite à la règle.
- Les outils ont échoué à plusieurs reprises.
- Le client demande expressément à parler à un responsable.

Après l'escalade :

- Donne l'ID de la demande.
- Indique le délai de réponse prévu.
- Remercie le client pour sa patience.

Style de communication

- Reste poli, professionnel et empathique en toute circonstance.
- Utilise le nom du client si tu le connais.
- Indique clairement les prochaines étapes.
- Reconnais les sentiments du client (agacement, inquiétude).
- Remercie le client pour sa patience.
- Ne fais jamais de promesses que tu ne peux pas tenir.

"""

`tools=[check_order_status, process_refund, escalate_to_supervisor]`

)

Points à noter :

Cet exemple illustre les principaux concepts évoqués dans cette partie :

1. Les **trois outils personnalisés** comportent des objectifs clairs et précis ainsi que des docstrings complètes.
2. Les **erreurs renvoyées sont structurées** avec la clé `status` et des valeurs `error_type` spécifiques.
3. Les **instructions stratégiques** sont divisées en plusieurs sections : capacités, workflows, gestion des exceptions, escalade.
4. Les instructions comportent des **consignes concernant le workflow séquentiel** : "COMMENCE par vérifier la commande, PUIS traite le remboursement."
5. La **gestion des exceptions est spécifique** à chaque type d'erreur (`not_found`,

`invalid_format, cannot_refund`).

6. Les **procédures d'escalade** sont clairement définies dans l'outil et dans les instructions.

Étape 3 : Exécuter et tester

`adk web`

Accédez à `http://localhost:8000` dans votre navigateur.

Étape 4 : Scénarios de test

Essayez ces scénarios de test pour voir comment des instructions stratégiques guident la coordination des outils et la gestion des exceptions :

Test 1 : Gestion des exceptions avec validation du format

Vous : `Vérifie la commande 123 pour moi.`

Résultat attendu : l'agent appelle `check_order_status("123")`, reçoit une erreur `invalid_format`, explique le format correct "ORD" avec un exemple et demande l'ID de commande corrigé.

Concept clé : des instructions spécifiques à chaque erreur permettent une réponse utile et éducative.

Test 2 : Workflow séquentiel multi-outils

Vous : `Je souhaite un remboursement pour la commande ORD789, car le produit n'a pas répondu à mes attentes.`

Résultat attendu :

1. L'agent appelle `check_order_status("ORD789")` pour vérifier l'existence et l'état de la commande.
2. Ensuite, il appelle `process_refund("ORD789", reason)` avec les informations sur la commande.
3. Enfin, il confirme le remboursement en indiquant le montant, le numéro de référence et le délai.

Concept clé : Les instructions doivent spécifier l'ordre dans lequel les outils sont utilisés. Il est important de vérifier AVANT d'agir.

Test 3 : Gestion des exceptions avec procédure d'escalade

Vous : Je souhaite me faire rembourser la commande ORD456. (Remarque : la commande ORD456 est "en cours de traitement", et non "livrée")

Résultat attendu : l'agent appelle `check_order_status`, puis `process_refund`, reçoit l'erreur `cannot_refund`, explique la règle (seules les commandes livrées peuvent être remboursées) et propose d'escalader à un superviseur.

Concept clé : Les instructions définissent les procédures d'escalade lorsque les outils ne permettent pas de résoudre le problème.

Points à retenir

Modèles de conception d'instructions :

- **Guide de sélection des outils** : "Utilise X lorsque le scénario Y se produit."
- **Gestion des résultats** : "Si status='success', alors... ; si une erreur est renvoyée, alors..."
- **Workflows séquentiels** : "Commence par utiliser X pour vérifier, puis Y pour agir."
- **Actions spécifiques à l'erreur** : le traitement diffère pour chaque type d'erreur.

Stratégie de gestion des exceptions :

- **Identifier les types d'erreurs** dans la conception des outils (`not_found`, `invalid_format`, `cannot_refund`)
- **Spécifier les actions** à mener pour chaque type d'erreur dans les instructions
- **Fournir des conseils** aux clients (expliquer, montrer des exemples, proposer des alternatives)
- **Définir les procédures d'escalade** pour les problèmes non résolus

Bonnes pratiques pour la coordination des outils :

- **Désigner les outils par leur nom** dans les instructions
- **Gérer tous les résultats** (succès, chaque type d'erreur, cas particuliers)
- **Séquencer** lorsque les outils doivent être utilisés dans un ordre précis
- **Définir les critères et les procédures d'escalade**

Modèle agent-as-tool :

- **Utiliser des agents spécialisés** pour des tâches de raisonnement complexes
- **Déléguer les sous-tâches** qui nécessitent une expertise du domaine
- **Séparer les tâches** (l'agent principal dirige, les agents spécialisés exécutent)
- **Importer AgentTool** : `from google.adk.tools.agent_tool import AgentTool`
- **Recommandé pour** les tâches nécessitant de faire preuve de jugement (en opposition avec les opérations déterministes)

Checklist pour la qualité :

Conception d'outils

- ✓ Noms de fonctions descriptifs (verbe-nom)
- ✓ Indications de type pour tous les paramètres
- ✓ Docstrings complètes (quoi, quand, arguments, valeurs de retour)
- ✓ Valeurs de retour incluant la clé d'état
- ✓ Types d'erreurs spécifiques dans les valeurs de retour

Conception des instructions

- ✓ Conseils sur la sélection des outils (quand utiliser chacun)
- ✓ Étapes du workflow (procédures séquentielles)
- ✓ Gestion des exceptions (actions pour chaque type d'erreur)
- ✓ Critères d'escalade (quand et comment)

Coordination

- ✓ Logique séquentielle définie
- ✓ Workflows multi-outils documentés
- ✓ Intégration des résultats spécifiée
- ✓ Consignes pour le style de communication